

TinyCoder: A 1.3B-Parameter Code Model with Sliding-Window Attention

Asha Raman, Daniel Okafor and Mei Lin (fictional authors)

FICTIONAL PAPER, written for this workshop

Abstract

We present TinyCoder, a 1.3-billion-parameter language model for Python code completion. Using sliding-window attention and 200 billion tokens of filtered Python code, TinyCoder comes within one point of a 7-billion-parameter baseline on our completion benchmark while running about 4× faster on a single GPU. We describe the architecture, the training data and the evaluation, and we report where the small model falls short.

1 Introduction

Large code models are accurate but expensive to serve. Every completion request in an editor has to return in a fraction of a second, and a 7B-parameter model needs a data-centre GPU to do that for many users at once. Small models are cheap to run but usually weaker.

Code completion is one of the most used features of a modern editor. Developers accept or reject a suggestion within a second or two, so a model that answers slowly is a model that is not used. A wrong suggestion also costs time, because the developer has to read it and undo it. Speed and accuracy both matter, and the trade-off between them is the subject of this paper.

We ask whether architecture and data choices can close the gap for a single programming language. Our answer is a narrow model: Python only, trained on carefully filtered code, with an attention pattern chosen for speed. This paper makes three contributions: a 1.3B-parameter model, a description of the data pipeline behind it, and an honest account of the cases where it is weaker than the larger baseline.

Section 2 describes the model, Section 3 the training data and the benchmark, Section 4 the results, and Sections 5 and 6 what they mean and where they stop.

Earlier small code models were usually trained on many languages at once, which spreads a small model's capacity thin. Local attention patterns have been used for documents in natural language, where most of the useful context is nearby. Source code has the same property: most of what a completion needs is in the lines just above the cursor.

This preprint describes the design and the measurements behind TinyCoder, written so that the trade-offs can be discussed. It does not come with released weights.

2 Methods

As its attention mechanism, TinyCoder uses sliding-window attention in every layer, with a 1,024-token window: each token attends only to the 1,024 tokens before it. We chose it for one reason, cost: with full attention the compute and memory needed grow quadratically with the length of the file, while with a fixed window they grow linearly.

In practice this means that doubling the size of a file only doubles the work. Information from further back still flows forward, because each layer passes it on to the next, but it arrives indirectly and weakens with distance.

The rest of the model is a plain decoder-only transformer: 24 layers, a hidden size of 2,048 and 16 attention heads, for 1.3 billion parameters in total. It uses the same tokenizer as the baseline, so the two models see identical inputs and their scores can be compared directly.

Why 1,024 tokens? Most Python functions in our data are shorter than 300 tokens, and most of the context a completion needs sits close to the cursor: the current function, its imports and the code just above it. A window of 1,024 tokens covers that in almost every case we inspected, while keeping the cost of each token small.

The key-value cache is bounded by the window too, so memory use at inference time stays flat however big the file grows. Together, the small size and the fixed window are what make TinyCoder fast on a single GPU.

Positions are encoded with rotary embeddings, and every feed-forward block has the same width. We kept the architecture deliberately plain, so that any difference in speed can be traced to the model size and the attention pattern rather than to custom GPU kernels.

At inference time the model generates one token at a time and stops at the end of the function body, or after 256 new tokens, whichever comes first.

The window size is a single setting, so it is easy to change. A smaller window makes the model faster still but misses more context; a larger one costs more per token. We kept the value above for every experiment in this paper.

3 Training and evaluation

TinyCoder was trained on 200B tokens of permissively licensed Python from public repositories. Training ran on 64 GPUs for 9 days.

The data was deduplicated before training. We also removed scripts under 50 tokens, scripts with more than 30% non-ASCII characters, code that looked auto-generated, and anything whose licence could not be identified. About 40% of the raw data survived, and notebooks were converted to plain scripts.

We evaluate on 2,000 held-out Python functions written by the authors. For each function the model sees the signature and the docstring and must write the body. We report `pass@1`: the share of functions whose first generated body passes all of its unit tests. The baseline is a 7B-parameter code model, evaluated with the same prompts and the same tests.

The benchmark covers string processing, data structures, numerical code, input and output, and small algorithms, in roughly equal numbers.

Each function comes with between three and twelve unit tests. A body that raises an exception, times out after ten seconds or fails any test counts as a miss. We sample once per function with greedy decoding, so the numbers are deterministic for a given model.

Every run of the benchmark uses the same hardware, one data-centre GPU, so that speed numbers are comparable between the two models. Speed is measured as tokens generated per second, averaged over all 2,000 functions.

We also recorded how often each model produced code that did not parse at all. Both models stayed below 2% on this measure, so we do not discuss it further.

The baseline was not retrained for this paper. We used its published checkpoint and ran it through exactly the same harness as TinyCoder, so that differences come from the models and not from the evaluation code. We report a single number per model rather than a range, because decoding is greedy. No test needs the network.

4 Results

TinyCoder solves 41% of functions on the first attempt (pass@1) versus 42% for the 7B baseline, with about 4× faster inference on the same GPU. Table 1 summarises the comparison.

The speed-up comes from two places: the model is small, and sliding-window attention keeps the cost of each new token constant.

Speed matters most for interactive use. At 2,400 tokens per second, TinyCoder finishes a typical 60-token completion in about 25 milliseconds on one GPU, against about 100 milliseconds for the baseline.

We checked whether the results depend on the order of the benchmark. Re-running both models on five shuffled orders changed pass@1 by less than half a point, so the one-point gap is stable.

Quality is not uniform across file sizes, as Table 1 also shows: on files longer than 1,024 tokens it drops to 31% versus 39% for the baseline. Long files are where the window hurts: code that depends on definitions far outside the window is harder to complete.

Errors on long files fall into a pattern. Most misses there are calls to helper functions defined earlier in the file, outside the window, whose names or arguments the model guesses wrongly. On short files, most misses are small slips, such as an off-by-one index or a wrong default value.

Model	Params	pass@1	tok/s	> 1,024 tok.
TinyCoder	1.3B	41%	2,400	31%
7B baseline	7B	42%	600	39%

Table 1: TinyCoder against the 7B baseline on 2,000 held-out functions. Last column: pass@1 on inputs over 1,024 tokens. tok/s: tokens generated per second on one GPU.

5 Discussion

Small, specialised models can be competitive for single-language completion. For an editor plugin, a model that is four times faster and one point less accurate is often the better trade, because latency is what users notice first.

The gap on files beyond the window is the clearest cost of the design. Retrieving definitions from elsewhere in the repository, or a wider window in the top layers, may close it; we leave both to future work.

For teams choosing a completion model, our results suggest a simple habit: measure speed and accuracy on your own code before defaulting to the larger model.

A second lesson concerns data. Most of the quality of a small model comes from what it is trained on: aggressive filtering and deduplication mattered more in our early experiments than any change to the architecture.

Finally, a fast model changes how completion feels. Suggestions that arrive while the developer is still typing get read; suggestions that arrive a second later are often ignored, however good they are.

6 Limitations

The authors see three limitations. Python only: TinyCoder was trained and tested on Python alone. Other languages, such as JavaScript and Java, were not evaluated. Whether it would work for JavaScript, Java or any other language is unknown. Languages differ in syntax, typing and library conventions, and a code model trained on one language usually completes others poorly, so a JavaScript or Java version of TinyCoder would need its own training data and its own benchmark.

One benchmark, built by the authors. All results come from our own 2,000 functions, which may favour the style of code we trained on, and tasks that span several files were not evaluated. A second, independent benchmark would make the comparison with the baseline much more convincing.

Contamination check. We checked for benchmark contamination, meaning benchmark functions that also appear in the training data, by exact-match overlap only. Near-duplicates with renamed variables would not be caught, so some contamination may remain.

References

- [1] A. Kumar and L. Chen. Sliding-window attention for efficient long-sequence transformers. Workshop on Efficient Models, 2024.
- [2] S. Ito and M. Brandt. Sliding-window attention revisited: what the window forgets. 2025.
- [3] P. Moreau. Sliding-window attention for code models on a single GPU. 2026.
- [4] J. Novak. Sliding-window attention versus full attention in small transformers. 2025.
- [5] R. Patel, S. Gupta and T. Wong. A code model for Python completion at scale. 2023.
- [6] T. Osei. Evaluating code models with unit tests: pass@k explained. 2022.
- [7] H. Silva, R. Costa and P. Lima. Deduplication of source code for language model training. 2024.
- [8] K. Haddad. Key-value cache management for small code models. 2025.
- [9] L. Fischer and A. Gomez. Contamination in code benchmarks: an exact-match study. 2024.
- [10] Y. Tanaka. A small code model for editor completion. 2025.
- [11] D. Mensah. Long-context code completion with sliding-window attention and a memory of earlier files. 2026.
- [12] E. Rossi, F. Bianchi and G. Conti. Permissive licences in public code datasets. 2023.